Dallen Teruel

March 4th, 2026

IT FDN 110 GP

Assignment 06

https://github.com/DallenTeruel/IntroToProg-Python-Mod06

Functions

Intro

In Module 06, I learned how to organize Python programs more effectively using functions, parameters, return values, and classes. Instead of relying on global variables, this module showed how passing data between functions makes programs easier to read, reuse, and maintain. I also learned how classes help group related code and how docstrings and static methods improve code organization and documentation.

Script Header

The program starts with a script header that explains the purpose of the file and includes the author's name, date, and revision information. This makes the code easier to understand and maintain (Figure 1.1).

```
# ------------------------------------------------------------------------------------------ #
# Title: Assignment06
# Desc: This assignment demonstrates using functions
# with structured error handling
# Change Log: (Who, When, What)
# Dallen Teruel, 03/04/26, Modify Script
# ------------------------------------------------------------------------------------------ #
```

Figure 1.1: Script header comments containing the program title, description, author name, and current date.

Imports

Next, The script imports Python's built-in json module so the program can read student data from a JSON file and save updates back to it (Figure 1.2).

```
import json
```

Figure 1.2: Python import statement makes Python's built-in JSON library available so that our program can work with JSON data.

Constants

Two constants are used in the program. One constant store the menu text that is displayed to the user, and the other stores the file name (Enrollments.json) where student registration data is saved. Using constants keeps important values in one place and makes the program easier to update (Figure 1.3).

```
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------
'''
FILE_NAME: str = "Enrollments.json"
```

Figure 1.3: Constant string values for the program menu and the JSON file name used to store enrollment data.

Variables

The program also initializes a variable to store the user's menu choice and a list to hold all student enrollment records (Figure 1.4).

```
# Define the Data Variables
menu_choice: str = ''  # Hold the choice made by the user.
students: list = []  # a table of student data
```

Figure 1.4: Variable initialization for menu choice and the list of all students

Class FileProcesser/Functions

Next, the FileProcessor class is responsible for handling all file-related tasks. One function reads student data from the JSON file and loads it into a Python list, while another function writes the updated list back to the file. Both functions include error handling to prevent crashes if the file is missing or if invalid data is encountered. This class demonstrates the Separation of Concerns principle by keeping file processing separate from user interaction (Figure 1.5-1.7).

```
class FileProcessor:  2 usages
    """
    A collection of processing layer functions that work with Json files

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created Class
    """
```

Figure 1.5: Class FileProcessor including doc strings

```python
# When the program starts, read the file data into table
# Extract the data from the file
# Read from the Json file
@staticmethod  1 usage
def read_data_from_file(file_name: str, student_data: list):
    """ This function reads student_data from JSON file

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created function to read student_data from JSON file

    """
    file = None

    try:
        file = open(file_name, "r")
        student_data = json.load(file)
    except FileNotFoundError as e:
        IO.output_error_messages( message: "Text file must exist before running this script!", e)
    except Exception as e:
        IO.output_error_messages( message: "There was a non-specific error!", e)
    finally:
        if file is not None and file.closed == False:
            file.close()
    return student_data
```

Figure 1.6: Function reads the student_data from JSON File

```python
@staticmethod  1 usage
def write_data_to_file(file_name: str, student_data: list):
    """ This function writes student_data to a JSON file

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created function

    """
    # global file
    # global students
    file = None

    try:
        file = open(file_name, "w")
        json.dump(student_data, file, indent=2)
    except TypeError as e:
        IO.output_error_messages( message: "Please check that the data is a valid JSON format", e)
    except Exception as e:
        IO.output_error_messages( message: "There was a non-specific error!", e)
    finally:
        if file is not None and file.closed == False:
            file.close()
```

Figure 1.7: Function writes the student_data from JSON File

Class IO/Functions

The IO class is responsible for all user interaction in the program, keeping input and output separate from the processing logic. It includes functions to display error messages, show

the menu, get the user's menu choice, display student registrations in CSV format, and collect student information such as first name, last name, and course name. Basic error handling is included in managing invalid user input. This class helps keep the program organized and easy to follow by grouping all presentation-related tasks into one place.

```
# Presentation ------------------------------------- #
class IO:  12 usages
    """
    A collection of presentation layer functions that manage user input and output
     ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created Class
    Dallen Teruel,3.4.2026,Added menu output and input functions
    Dallen Teruel,3.4.2026,Added a function to display the data
    Dallen Teruel,3.4.2026,Added a function to display custom error messages
    """
```

Figure 1.8: Collection of presentation layer functions that manage user input and output

```
@staticmethod  7 usages
def output_error_messages(message: str, error: Exception = None):
    """ This function displays the custom error messages to the user

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created function

    :return: None
    """
    print(message, end="\n\n")
    if error is not None:
        print("-- Technical Error Message -- ")
        print(error, error.__doc__, type(error), sep='\n')
```

Figure 1.9: Function displays the custom error messages to the user

```python
@staticmethod  1 usage
def output_menu(menu: str):
    """ This function displays the menu of choices to the user

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created function

    :return: None
    """
    print()
    print(menu)
    print()  # Adding extra space to make it look nicer.
```

Figure 1.10: Function displays the menu of choices 1-4 to the user

```python
@staticmethod  1 usage
def input_menu_choice():
    """ This function gets a menu choice from the user

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created function

    :return: string with the users choice
    """
    choice = "0"
    try:
        choice = input("Enter your menu choice number: ")
        if choice not in ("1", "2", "3", "4"):  # Note these are strings
            raise Exception("Please, choose only 1, 2, 3, or 4")
    except Exception as e:
        IO.output_error_messages(e.__str__())  # Not passing the exception object to avoid the technical message
    return choice
```

Figure 1.11: Function gets the menu choice from the user (1-4)

```python
@staticmethod  2 usages
def output_student_courses(student_data: list) -> None:
    """
    Displays current student registrations as comma-separated values:
    FirstName,LastName,CourseName

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created function

    """
    print("-" * 50)
    print(f"Current Student Registrations:")
    for row in student_data:
        print(f'{row["FirstName"]},{row["LastName"]},{row["CourseName"]}')
    print("-" * 50)
```

Figure 1.12: Displays current student registrations in CSV format

```python
@staticmethod  1 usage
def input_student_data(student_data: list):
    """ This function gets the first name, last name, and course name

    ChangeLog: (Who, When, What)
    Dallen Teruel,3.4.2026,Created function

    :return: None
    """

    try:
        # Input the data
        student_first_name = input("What is the student's first name? ")
        if not student_first_name.isalpha():
            raise ValueError("The first name should not contain numbers.")

        student_last_name = input("What is the student's last name? ")
        if not student_last_name.isalpha():
            raise ValueError("The last name should not contain numbers.")

        course_name = input("Please enter the name of the course: ")
        # Add to student "two-dimensional list of dictionary rows"
        student = {"FirstName": student_first_name,
                   "LastName": student_last_name,
                   "CourseName": course_name}
        student_data.append(student)
        print(f"\nYou have registered {student_first_name} {student_last_name} for {course_name}.")

    except ValueError as e:
        IO.output_error_messages( message: "That value is not the correct type of data!", e)
    except Exception as e:
        IO.output_error_messages( message: "There was a non-specific error!", e)
    return student_data

End of function definitions
```

Figure 1.13: Function gets the students first name, last name and course name

Input/Output/Processing

Lastly, when the program starts, it loads any existing data from the JSON file. A while loop then keeps the program running until the user chooses to exit.

- Option 1 allows the user to register a student by entering a first name, last name, and course name.
- Option 2 displays all current student registrations.

- Option 3 saves the data to the JSON file and confirms the save.
- Option 4 exits the program and ends the loop (Figure 1.14).

```python
# Beginning of the main body of this script
students = FileProcessor.read_data_from_file(FILE_NAME, students)

# Repeat the following tasks
while True:
    IO.output_menu(menu=MENU)
    menu_choice = IO.input_menu_choice()

    if menu_choice == "1":  # Prompt to input Student Registration Data
        IO.input_student_data(student_data=students)
        continue

    elif menu_choice == "2":  # Display comma-separated values for each row
        IO.output_student_courses(student_data=students)
        continue

    elif menu_choice == "3":   # Save data to file, then display what was written using students variable
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        print("\nThe following data was saved to file!")
        IO.output_student_courses(students)
        continue

    elif menu_choice == "4":  # End the program
        break  # out of the while loop
```

Figure 1.14: Main Body loads saved data, repeatedly shows a menu, performs the selected action on the student list, and runs until the user chooses to exit.

Test

The program was tested in both IDLE and the Windows Command Prompt. All menu options worked correctly, including registering students, displaying data, saving to the file, handling invalid input, and exiting the program (Figures 1.15–1.20).

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 1
What is the student's first name? Dallen
What is the student's last name? Teruel
Please enter the name of the course: Python 100

You have registered Dallen Teruel for Python 100.
```

```
Enter your menu choice number: 1
What is the student's first name? Vic
What is the student's last name? Vu
Please enter the name of the course: Python 100

You have registered Vic Vu for Python 100.
```

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 1
What is the student's first name? Dallen
What is the student's last name? Teruel
Please enter the name of the course: Python 100

You have registered Dallen Teruel for Python 100.
```

```
Enter your menu choice number: 1
What is the student's first name? Vic
What is the student's last name? Vu
Please enter the name of the course: Python 100

You have registered Vic Vu for Python 100.
```

Figure 1.15: The user registers two students for a course. The user first selects menu option 1 and enters each student's first name, last name, and course name to create individual enrollment records.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 2
--------------------------------------------------
Current Student Registrations:
Bob,Smith,Python 100
Sue,Jones,Python 100
Dallen,Teruel,Python 100
Vic,Vu,Python 100
--------------------------------------------------
```

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 2
--------------------------------------------------
Current Student Registrations:
Bob,Smith,Python 100
Sue,Jones,Python 100
Dallen,Teruel,Python 100
Vic,Vu,Python 100
--------------------------------------------------
```

Figure 1.16: The user selects menu option 2 to display the current registration data. The program outputs both student records in a format, comma-separated format that includes each student's first name, last name, and course name.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 3

The following data was saved to file!
--------------------------------------------------
Current Student Registrations:
Bob,Smith,Python 100
Sue,Jones,Python 100
Dallen,Teruel,Python 100
Vic,Vu,Python 100
--------------------------------------------------
```

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 3

The following data was saved to file!
--------------------------------------------------
Current Student Registrations:
Bob,Smith,Python 100
Sue,Jones,Python 100
Dallen,Teruel,Python 100
Vic,Vu,Python 100
--------------------------------------------------
```

Figure 1.17: The user selects menu option 3, which saves the registration data to the Enrollments.json file. A confirmation message indicating that the data was saved successfully is displayed.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 9
Please, choose only 1, 2, 3, or 4
```

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 9
Please, choose only 1, 2, 3, or 4
```

Figure 1.18: The user enters an invalid menu option (5), and the program displays a message instructing the user to choose an option between 1 and 4.

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 4

Process finished with exit code 0
```

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 4

C:\Users\chq-dallent\Documents\Python\PythonCourse\Module06>
```

Figure 1.19: The user selects menu option 4, which exits the program and displays a program termination message.

Figure 1.20: The Enrollments.json file displaying multiple student registration records stored in valid JSON format.

Source Control

- Link to GitHub repository
- Screenshot of link to GitHub repository in GitHub links forum

DT

Dallen Teruel

Mar 4 6:54pm

Here is my GitHub link for Assignment #6:

https://github.com/DallenTeruel/IntroToProg-Python-Mod06

Reply | Like | Mark as Unread

Summary

In Module 06, overall, this module helped me write cleaner and more organized Python programs. Using functions, classes, and the Separation of Concerns approach made the code easier to read, test, and maintain. This structure also makes it easier to extend the program in the future.